

Stages of bootstrapping

In what follows we denote the provenance of the components generated by a scalalr processor from a dialect of the scalalr input language by:

```
(bootstrap.Syntax, bootstrap.Generator)(scalalrInputFile)
```

This denotes the components generated from `scalalrInputFile` by the bootstrap code-generator. The `scalalrInputFile` must be in the notation accepted by the handwritten bootstrap parser. The generated components are always placed in a directory specified in the prologue of the input file.

Stage 1: bootstrap

The bootstrapping sequence for `scalalr` starts with the bootstrap generator constructed entirely by hand in `scala`. This is specified entirely (ie input language syntax and output component semantics) by objects of the `bootstrap` module, and the driver for the runnable program is `scalalrlifeboat.scala`. We put these all together (from the `LIFEBOAT/` directory) using `sbt` to package the dependencies for the bootstrap, then `scala-cli` to construct a self-contained runnable for eventual placement in the `scripts/` directory.

```
(cd ../; sbt "bootstrap / clean ; bootstrap / package")
```

```
scala-cli --power package -o scalalrlifeboat --assembly -f scalalrlifeboat.scala
```

The runnable `scalalrlifeboat` makes parser components for any notation described in the bootstrap `.scalalr` notation. The generated components can be incorporated in any program that needs a parser, and this parser is implemented by the `shared` module parsing automaton `LRParser`.

An extract from such a program `runtinyfun` – all of whose components can be found in the `LIFEBOAT/` directory – may be helpful in illustrating how straightforward the construction of a program that incorporates scalalr components can be.

```
object runtinyfun {
  import org.sufrin.scalalr._
  import org.sufrin.utility.SourceTextCursor
  import java.nio.file.Paths

  import tinyfun.Components
  import tinyfun.Scanner._

  val log = args.contains("-l")
  val file = (args.toList.filterNot(_.startsWith("-")) ++ List("/dev/tty")).head

  print("Welcome to TinyFun\n> ")
}
```

```

val scanner = Scanner(SourceTextCursor(Paths.get(file)))
val parser = LRParser.Pull[Token](Components)(scanner.sourceLocation)
parser.logState = log
try parser.run(scanner.next) catch {
  case err: java.lang.Error => println(err)
}
}

```

Its parser components are generated (below the `generated/` directory) by

```
scalalrlifeboat --output=generated tinyfun.scalalr
```

their provenance is

```
(bootstrap.Syntax, bootstrap.Generator) (tinyfun.scalalr)
```

The `generated/` components, and the remaining essential components of `runtinyfun` – its driver program `runtinyfun.scala` and its abstract syntax and semantics defined in `TinyFun.scala` – can be put together using `scala-cli`:

```

scala-cli compile runtinyfun.scala TinyFun.scala generated
scala-cli run runtinyfun.scala TinyFun.scala generated

```

or compiled and packaged into a self-contained script by

```
scala-cli --power package runtinyfun.scala TinyFun.scala generated -o runtinyfun --assembly
```

The really useful thing about `scala-cli` is that it makes very straightforward the declaration of the dependencies of a source file. The whole of `runtinyfun.scala` is:

```

//> using scala 2.13
//> using jar ../bootstrap/target/bootstrap-0.8.0.jar
//> using jar ../shared/target/shared-0.8.0.jar
//> using jar ../utilities/target/utilities-0.8.0.jar
//> using jar ../logging-api/lib/Logging.jar
//> using dep org.scala-lang.modules::scala-xml::2.4.0

object runtinyfun {
  ...
}

```

Stage 2: FLaB

The next stage (First Language atop Bootstrap) is the first stage at which `scalalr`-generated parser components can be incorporated into a processor for a dialect of the `scalalr` notation. There are two possibilities for building generator components. The first yields:

```
flab = (bootstrap.Syntax, bootstrap.Generator)(flab-notation.scalalr)
```

the second uses the `LRParser` automaton with the `flab` components to parse the notation it was described in, then the `flab.Generator` to generate the corresponding components:

```
(LRParser(flab), flab.Generator)(flab-notation.scalalr)
```

In fact the `flab.Generator` does no more than translate `flab` abstract syntax into bootstrap syntax, then invoke the `bootstrap.Generator`.

So when compiled *alongside the rest of the FLaB implementation*, namely `Generator.scala` and `AST.scala` in the appropriate `flab/src/main/scala` directory, this yields a generator *with the same component semantics and the same input language syntax* but whose parser is now implemented with the `LRParser` shared automaton and the generated components. (See `PREPAREFLAB/` for an account of this)

Flab vs Bootstrap Syntax

Although the handwritten bootstrap parser accepts a slightly more general notation than the Scalalr-generated parser for Flab, there is no practically-useable difference between them.

Further stages: SLaB ...

Further stages - starting with *SLaB* - will incorporate a re-engineered component generation phase, and also be self-hosting. Our intention is also to test some macro-like novelties that support more concise descriptions of list- and option-yielding productions.